软件工程课程设计

校园快递收发系统

系统设计文档

System Design Document

专业班级：_____

小组编号：_____

小组成员：_____

指导教师：穆海伦

2026 年 10 月

文档信息

文档名称	校园快递收发系统系统设计文档	文档编号	CE-DESIGN-001
版本	V1.0	密级	内部
编写	项目组	审核	_____
日期	2026-10-26	状态	评审稿

修订记录

版本	日期	修订人	修订说明
V1.0	2026-10-26	项目组	初稿：总体结构、模块划分、接口设计、数据库设计、详细设计与部署设计

目　录

1 引言 1

1.1 编写目的 1

1.2 项目背景 1

1.3 术语与缩略语 1

1.4 参考资料 2

1.5 文档结构 2

2 系统概述 3

2.1 建设目标 3

2.2 用户角色 3

2.3 业务范围与边界 3

2.4 设计约束 4

2.5 运行环境 4

3 总体设计 5

3.1 设计原则 5

3.2 技术架构 6

3.3 模块结构图 8

3.4 模块清单与职责 9

3.5 核心业务流程 9

3.6 权限矩阵 10

4 接口设计 11

4.1 设计规范 11

4.2 接口清单 11

4.2.1 认证与用户（M1） 11

4.2.2 驿站与货架（M2） 12

4.2.3 快递入库与出库（M3、M4） 12

4.2.4 寄件管理（M5） 13
4.2.5 异常、通知、公告与统计（M6—M8） 13
4.3 关键接口定义 13
4.3.1 登录 13
4.3.2 快递入库 14
4.3.3 取件核验 15
4.3.4 寄件下单 15
4.4 错误码 16
4.5 接口安全与幂等 17
5 数据库设计 18
5.1 设计策略 18
5.2 E-R 图 19
5.3 实体关系说明 20
5.4 数据库表结构 20
5.4.1 users（用户表） 20
5.4.2 stations（驿站表） 21
5.4.3 shelves（货架表） 21
5.4.4 packages（快递包裹表） 21
5.4.5 pickup_records（取件记录表） 22
5.4.6 shipment_orders（寄件订单表） 23
5.4.7 shipment_tracks（寄件轨迹表） 23
5.4.8 notifications（通知表） 24
5.4.9 announcements（公告表） 24
5.4.10 exception_records（异常记录表） 25
5.4.11 operation_logs（操作日志表） 25
5.5 数据字典 25

5.6 索引与约束说明 26
6 详细设计 27
6.1 入库登记模块（M3） 27
6.1.1 功能说明 27
6.1.2 程序流程图 28
6.1.3 N-S 图 29
6.1.4 关键处理逻辑 30
6.1.5 异常处理 30
6.2 取件核验模块（M4） 30
6.2.1 功能说明 30
6.2.2 程序流程图 32
6.2.3 N-S 图 33
6.2.4 关键处理逻辑 34
6.2.5 异常处理 34
6.3 其他模块关键逻辑 35
7 界面设计 36
7.1 页面结构 36
7.2 路由与权限 36
7.3 关键交互 36
8 部署与运行设计 38
8.1 部署拓扑 38
8.2 Docker 编排设计 38
8.3 本地开发启动 39
8.4 配置项 40
8.5 初始化与迁移 40
9 非功能性设计 42

9.1 安全设计 42
9.2 性能设计 42
9.3 可靠性与容错 42
9.4 日志与审计 43
9.5 兼容性 43
10 交付物与实施计划 44
10.1 课程交付物对照 44
10.2 仓库目录 44
10.3 后续阶段计划 45
附录 A 包裹状态机 46
附录 B 设计待确认事项 47

提示：目录为 Word 域，首次打开如未显示页码，请全选后按 F9 或右键选择“更新域”。

1 引言

1.1 编写目的

本文档是“校园快递收发系统”的系统设计说明书，用于明确系统的总体结构、功能模块划分、接口规范、数据库结构与详细处理逻辑，为后续编码实现、系统测试、项目验收提供统一依据。

本文档的预期读者包括：课程指导教师、项目组全体成员，以及后续参与维护的开发人员。文档同时作为软件工程课程设计“系统设计”阶段的提交物，对应任务书中总体结构设计、数据库设计与详细设计三项要求。

1.2 项目背景

随着高校快递业务量持续增长，校园快递的收发逐渐由分散代收转向驿站集中管理。目前多数校园驿站仍采用“人工登记 + 纸质台账 + 微信群通知”的方式，存在以下问题：

1. 到站登记依赖手工录入，运单号易抄错，高峰期排队严重。
2. 取件码由工作人员口头或截图发送，缺少统一生成与失效机制，存在冒领风险。
3. 超期件缺少自动识别与提醒，货架长期被占用，驿站容量紧张。
4. 寄件业务与收件业务分离，学生无法在线查看寄件进度。
5. 收发量、超期率等数据依靠人工统计，无法支撑驿站工作量评估。

本系统面向校园场景，将快递到站、上架、通知、取件核验、超期处理与寄件下单整合到一个平台中，按“多驿站”模型组织数据，使学生、驿站工作人员与管理员在同一系统内完成各自工作。

1.3 术语与缩略语

表 1-1 术语与缩略语

术语	说明
驿站	校园内集中存放与发放快递的服务点，本系统支持多个驿站
运单号	快递公司生成的唯一物流单号，是包裹的全局标识
取件码	包裹入库后生成的 6 位数字凭证，在所属驿站内唯一
格口	货架上的最小存放单元，系统以“货架 + 容量”管理格口占用
超期件	入库后超过规定天数仍未取件的包裹，默认阈值为 3 天

术语	说明
异常件	破损、丢失、错分等需要人工登记处理的包裹
JWT	JSON Web Token，本系统用于无状态身份认证
RBAC	基于角色的访问控制，系统划分学生、驿站工作人员、管理员三类角色
ORM	对象关系映射，本系统使用 SQLAlchemy 2.0
SPA	单页应用，前端使用 Vue 3 构建
幂等	同一请求重复提交不产生重复业务结果

1.4 参考资料

1. 《软件工程课程设计》任务书与课程 PPT，2026 年秋季学期。
2. GB/T 8567—2006《计算机软件文档编制规范》。
3. FastAPI 官方文档：https://fastapi.tiangolo.com
4. Vue 3 官方文档：https://cn.vuejs.org
5. MySQL 8.0 Reference Manual：https://dev.mysql.com/doc/refman/8.0/en/
6. SQLAlchemy 2.0 Documentation：https://docs.sqlalchemy.org/en/20/

1.5 文档结构

本文档共 10 章。第 1 章说明编写目的与背景；第 2 章描述系统目标、角色与边界；第 3 章给出总体结构、模块结构图与权限矩阵；第 4 章定义接口规范与接口清单；第 5 章给出 E-R 图与全部数据库表结构；第 6 章对两个核心模块进行详细设计，给出程序流程图与 N-S 图；第 7 章说明界面与路由设计；第 8 章说明 Docker 与本地两种部署方式；第 9 章说明非功能性设计；第 10 章对照课程交付物并给出实施计划。附录给出包裹状态机与待确认事项。

2 系统概述

2.1 建设目标

系统面向校园快递收发场景，构建一个覆盖“到站—入库—上架—通知—取件—出库”收件闭环，并支持学生在线寄件的管理平台。具体目标如下：

1. 以运单号为唯一标识完成快递到站登记，入库操作平均耗时控制在 30 秒以内。
2. 入库时自动生成取件码并推送通知，取件时通过“取件码 + 手机号后四位”双重核验，降低冒领风险。
3. 自动识别超期件并提醒，支持退回登记，释放货架容量。
4. 支持多驿站、多货架的容量管理，实时展示各驿站占用情况。
5. 提供收发量、超期率、驿站工作量等统计报表，为驿站管理提供数据支撑。
6. 通过 Docker Compose 统一数据库与运行环境，保证小组内成员一键复现。

2.2 用户角色

系统设置三类角色，采用 RBAC 模型控制功能访问。

表 2-1 用户角色

角色	标识	典型用户	主要职责与权限
学生 / 教职工	student	在校学生、教师	查看本人包裹与取件码、在线寄件、查看寄件轨迹、接收通知、浏览公告
驿站工作人员	staff	驿站值班人员	快递入库、上架调整、取件核验出库、寄件揽收、异常件登记与退回处理
系统管理员	admin	系统管理教师、组长	用户与角色管理、驿站与货架维护、公告发布、全局统计、操作日志审计

说明：一个用户只能属于一个角色；驿站工作人员通过 `stations.manager_id` 或后续扩展的员工-驿站关联表确定所属驿站，本期设计中每名工作人员归属一个驿站。

2.3 业务范围与边界

系统包含以下业务范围：

1. 收件业务：到站登记、批量导入、货架分配、取件码生成、取件通知、取件核验、出库签收、超期判定、退回登记。
2. 寄件业务：在线下单、运费估算、驿站揽收称重、轨迹记录、订单完成与取消。

3. 基础数据：用户、驿站、货架、公告、通知。

4. 统计分析：收发量趋势、驿站工作量、超期率、异常件统计。

本期不包含以下内容，避免范围蔓延：

1. 不对接快递公司开放平台，运单信息由驿站工作人员录入或批量导入。

2. 不接入真实短信网关，通知以站内信为主，短信通道以日志模拟。

3. 不包含在线支付，寄件运费仅在系统中记录，线下结算。

4. 不包含跨校区调拨、无人车配送、智能柜硬件对接。

5. 不包含代取授权与费用结算，代取场景由工作人员线下核验处理。

2.4 设计约束

表 2-2 设计约束

类别	约束说明
技术栈	前端 Vue 3 + Vite + Element Plus；后端 Python 3.12 + FastAPI；数据库 MySQL 8.0
数据库部署	数据库必须运行在 Docker 容器中，使用官方镜像，保证组内环境一致
启动方式	同时支持本地开发启动（前后端热更新 + 容器化数据库）与 Docker Compose 一键启动
团队规模	4—5 人，按角色分工，需支持并行开发
开发周期	第 8 周至第 15 周，共 8 周开发与测试时间
运行环境	服务端 Linux / Windows 均可；客户端为现代浏览器，不需兼容 IE

2.5 运行环境

表 2-3 运行环境

项目	最低要求	推荐配置
客户端	Chrome 100+ / Edge 100+，1366×768	Chrome 最新版，1920×1080
服务端	2 核 4 GB，Docker 24+，Docker Compose v2	4 核 8 GB，SSD
开发环境	Python 3.12、Node.js 20+、Git 2.40+	同上，另装 VS Code 与 MySQL 客户端
数据库	MySQL 8.0（Docker 镜像 mysql:8.0）	同上，字符集 utf8mb4
缓存	Redis 7（Docker 镜像 redis:7-alpine）	同上

3 总体设计

3.1 设计原则

1. **分层解耦**：前端只通过 HTTP 接口访问后端，后端按“路由—服务—模型”三层组织，禁止在路由中直接写复杂业务逻辑。
2. **模块高内聚**：按业务域划分 8 个模块，每个模块拥有独立的 ORM 模型、Schema 与路由文件，模块之间通过服务层调用。
3. **接口先行**：先定义接口契约（路径、入参、出参、错误码），前后端可并行开发，前端使用 Mock 数据联调。
4. **数据库可迁移**：所有表结构变更通过 Alembic 迁移脚本管理，禁止手工改表后不入库。
5. **环境可复现**：数据库与缓存统一使用 Docker 镜像，配置文件通过环境变量注入，仓库只提交 `.env.example`。
6. **安全默认**：除登录、注册与健康检查外，所有接口默认要求 JWT；数据查询默认按角色限定范围，越权访问返回 403。

3.2 技术架构

系统采用前后端分离的 B/S 架构，自上而下分为用户层、接入层、前端层、接口层、业务层与数据层。

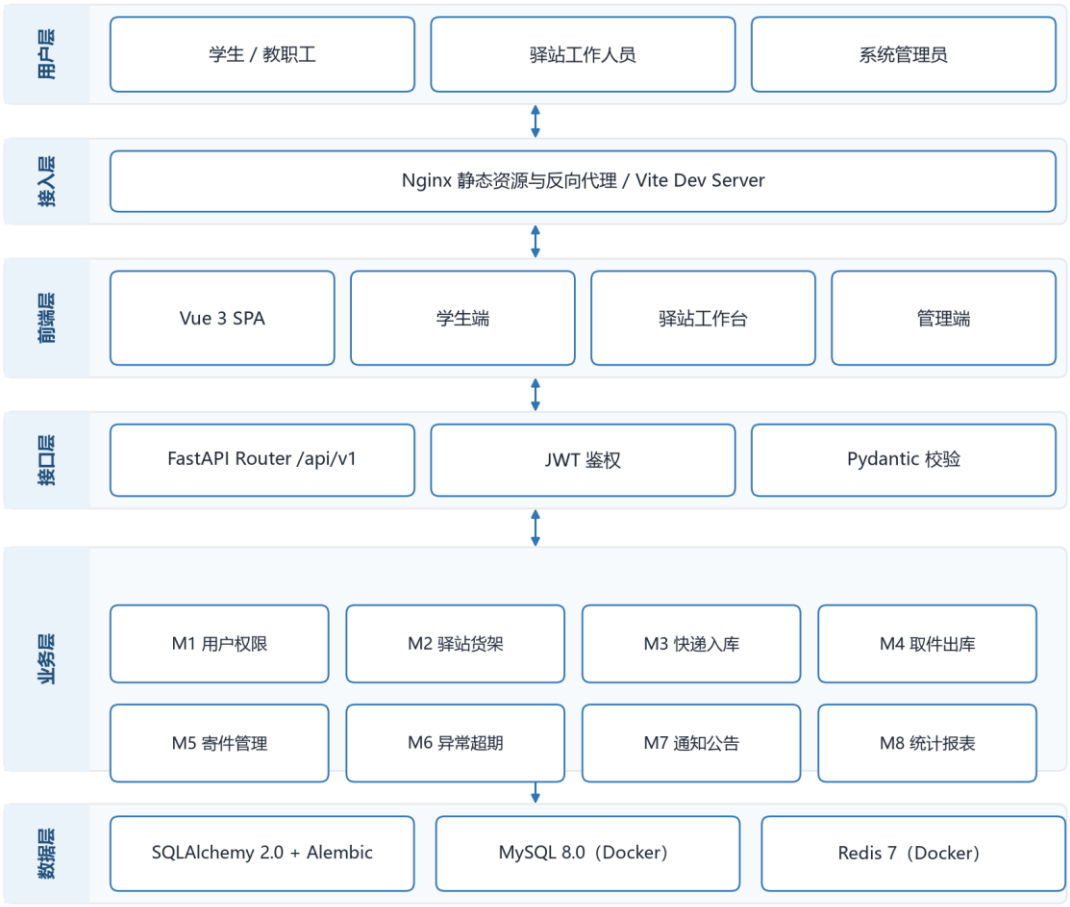


图 3-1 系统技术架构图

各层职责如下：

表 3-1 技术架构

层次	组成	职责
用户层	学生、驿站工作人员、管理员	通过浏览器访问系统，不同角色进入不同工作台
接入层	Nginx（生产）/ Vite Dev Server（开发）	静态资源托管、`/api` 反向代理、Gzip 压缩
前端层	Vue 3 + Vue Router + Pinia + Element Plus	页面渲染、路由守卫、状态管理、接口调用
接口层	FastAPI Router + Pydantic + JWT	路由分发、参数校验、身份认证、统一异常处理
业务层	8 个业务模块的 Service	业务规则、事务控制、权限校验、日志记录
数据层	SQLAlchemy 2.0 + Alembic + MySQL + Redis	数据持久化、迁移管理、取件码与令牌缓存

前端通过 Axios 调用 `/api/v1` 下的接口，请求头携带 `Authorization: Bearer <token>`；后端在依赖注入阶段解析令牌并校验角色。Redis 用于缓存取件码查询结果与登录黑名单，降低高频取件场景下的数据库压力。

3.3 模块结构图

按照“系统—功能模块—子功能”三级结构，系统划分为 8 个功能模块，模块结构如图 3-2 所示。

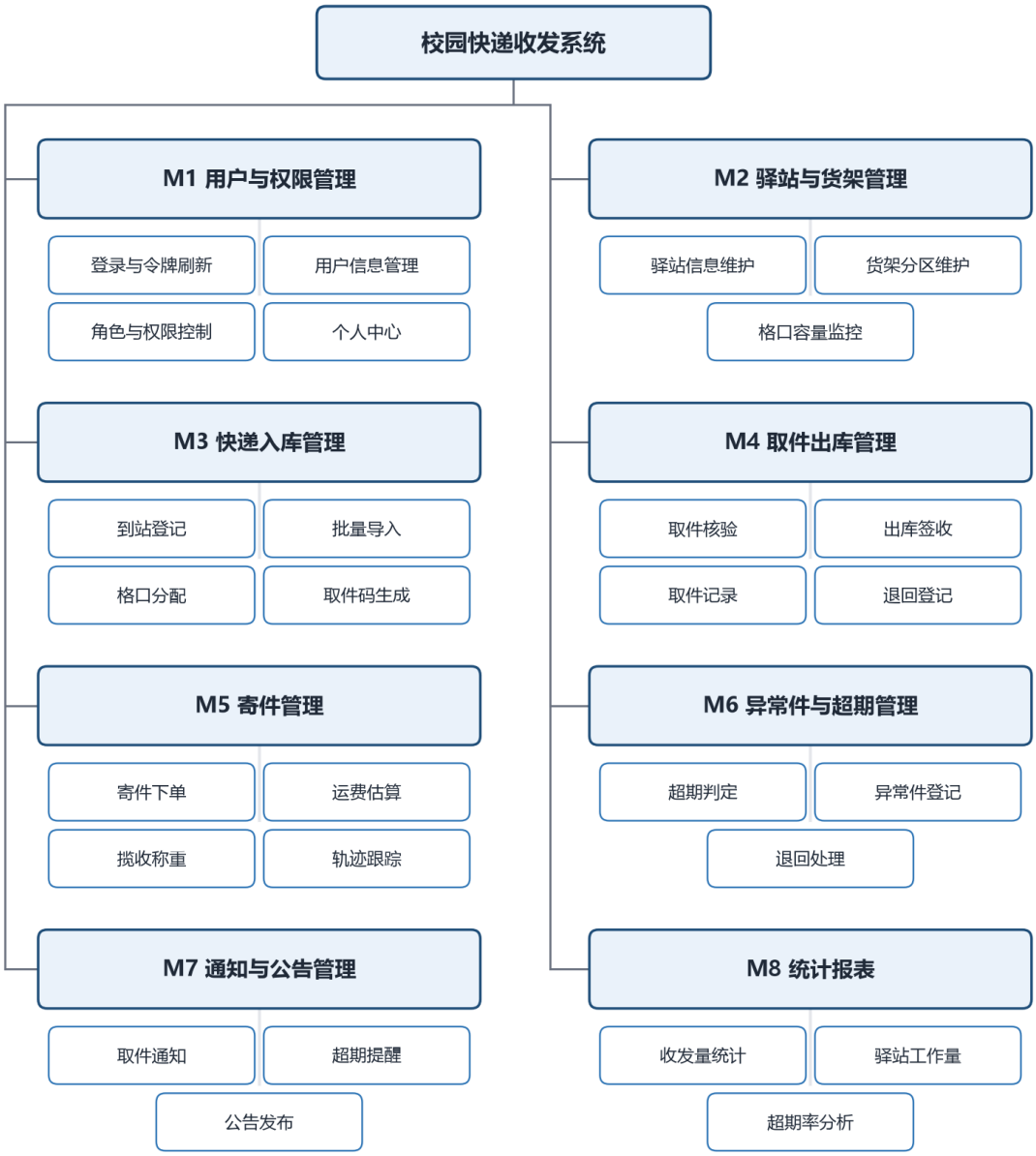


图 3-2 系统模块结构图

3.4 模块清单与职责

表 3-2 模块清单与职责

编号	模块名称	主要职责	子功能	涉及数据表
M1	用户与权限管理	账号、认证、角色与个人信息管理	登录与令牌刷新、用户信息管理、角色与权限控制、个人中心	users
M2	驿站与货架管理	驿站档案、货架分区与容量维护	驿站信息维护、货架分区维护、格口容量监控	stations、shelves
M3	快递入库管理	到站登记、批量导入、格口分配与取件码生成	到站登记、批量导入、格口分配、取件码生成	packages、shelves
M4	取件出库管理	取件核验、出库签收、取件记录与退回	取件核验、出库签收、取件记录、退回登记	packages、pickup_records
M5	寄件管理	寄件下单、运费估算、揽收称重与轨迹跟踪	寄件下单、运费估算、揽收称重、轨迹跟踪	shipment_orders、shipment_tracks
M6	异常件与超期管理	超期判定、异常登记与退回处理	超期判定、异常件登记、退回处理	packages、exception_records
M7	通知与公告管理	取件通知、超期提醒与公告发布	取件通知、超期提醒、公告发布	notifications、announcements
M8	统计报表	收发量、驿站工作量与超期率统计	收发量统计、驿站工作量、超期率分析	多表聚合

模块之间的主要依赖关系为：M3、M4 依赖 M2 提供的驿站与货架数据；M4 完成后由 M7 推送取件成功通知；M6 的超期判定基于 M3 写入的入库时间；M8 读取 M3—M6 产生的业务数据做聚合，不反向修改业务数据。

3.5 核心业务流程

收件闭环：

1. 快递到达驿站，工作人员扫描或输入运单号，系统校验运单号唯一性。
2. 系统按收件人手机号匹配用户，未匹配则登记为临时收件人。
3. 工作人员选择驿站，系统推荐空闲格口并完成上架。
4. 系统生成 6 位取件码，写入包裹记录，状态置为“在库”。
5. 系统向收件人推送取件通知，通知中包含取件码、驿站地址与营业时间。
6. 收件人到站出示取件码与手机号后四位，工作人员录入取件码。
7. 系统核验取件码、手机号与包裹状态，通过后写入取件记录，状态置为“已取件”。
8. 系统释放货架格口，推送取件成功通知；若为超期件，同时记录超期天数。

寄件流程：

1. 学生在线填写寄件信息，选择驿站，系统按重量区间估算运费。
2. 生成寄件订单，状态为“待揽收”。
3. 工作人员在驿站揽收，称重确认实际运费，状态置为“已揽收”。
4. 包裹发出后登记轨迹节点，状态置为“已发出”。
5. 收件人签收后，工作人员确认完成，状态置为“已完成”。
6. 学生可随时查看订单状态与轨迹节点。

3.6 权限矩阵

表中“√”表示可操作，“只读”表示仅可查看本人或本站数据，“—”表示无权限。

表 3-3 权限矩阵

功能	学生	驿站工作人员	管理员
登录、修改本人密码	√	√	√
查看本人包裹与取件码	√	—	只读
快递入库、批量导入	—	√	√
取件核验与出库	—	√	√
退回与异常登记	—	√	√
寄件下单	√	—	—
寄件揽收、轨迹登记	—	√	√
驿站、货架维护	—	只读	√
用户与角色管理	—	—	√
公告发布	只读	只读	√
统计报表	—	本站	全局
操作日志审计	—	—	√

4 接口设计

4.1 设计规范

系统接口遵循 RESTful 风格，统一前缀为 `/api/v1`，请求与响应均使用 JSON（文件上传除外）。

表 4-1 设计规范

项目	规范
基础路径	`/api/v1`
资源命名	使用复数名词，小写中划线，如 `/pickup-records`
请求方法	GET 查询、POST 创建或动作、PUT 全量更新、PATCH 局部更新、DELETE 删除
认证方式	请求头 `Authorization: Bearer <access_token>`
内容类型	`application/json; charset=utf-8`
时间格式	ISO 8601，如 `2026-10-26T18:30:00+08:00`
分页参数	`page`（默认 1）、`page_size`（默认 20，最大 100）
排序参数	`sort`，如 `sort=-created_at` 表示按创建时间倒序

统一响应结构：

```
{
  "code": 0,
  "message": "success",
  "data": {},
  "request_id": "c1f7a2e4-9b3d-4a6e-8f21-5d0c7b9e4a10"
}
```

分页响应的 `data` 结构：

```
{
  "items": [],
  "total": 128,
  "page": 1,
  "page_size": 20
}
```

HTTP 状态码与业务码同时使用：HTTP 状态码表达通信层结果，`code` 表达业务结果。成功时 `code` 为 0；失败时 `code` 为具体业务错误码，`message` 为可直接展示的中文提示。

4.2 接口清单

4.2.1 认证与用户（M1）

表 4-2 认证与用户（M1）

方法	路径	说明	权限

方法	路径	说明	权限
POST	/auth/register	学生注册，手机号唯一	公开
POST	/auth/login	账号密码登录，返回双令牌	公开
POST	/auth/refresh	使用刷新令牌换取新的访问令牌	公开
POST	/auth/logout	注销，令牌加入黑名单	登录用户
GET	/auth/me	获取当前登录用户信息	登录用户
PUT	/auth/password	修改本人密码	登录用户
GET	/users	分页查询用户，支持角色、关键字筛选	管理员
POST	/users	新增用户（工作人员、管理员）	管理员
GET	/users/{id}	查看用户详情	管理员
PUT	/users/{id}	修改用户信息	管理员
PATCH	/users/{id}/status	启用或禁用账号	管理员

4.2.2 驿站与货架（M2）

表 4-3 驿站与货架（M2）

方法	路径	说明	权限
GET	/stations	驿站列表，支持状态筛选	登录用户
POST	/stations	新增驿站	管理员
GET	/stations/{id}	驿站详情与容量统计	登录用户
PUT	/stations/{id}	修改驿站信息	管理员
PATCH	/stations/{id}/status	启用或停用驿站	管理员
GET	/stations/{id}/shelves	查询驿站下的货架及占用率	工作人员、管理员
POST	/stations/{id}/shelves	新增货架	工作人员、管理员
PUT	/shelves/{id}	修改货架信息	工作人员、管理员
DELETE	/shelves/{id}	删除无包裹的货架	管理员

4.2.3 快递入库与出库（M3、M4）

表 4-4 快递入库与出库（M3、M4）

方法	路径	说明	权限
GET	/packages	分页查询包裹，支持状态、驿站、关键字筛选	工作人员、管理员
POST	/packages	单件入库，生成取件码	工作人员、管理员
POST	/packages/batch	批量入库，返回逐条结果	工作人员、管理员
GET	/packages/{id}	包裹详情与流转记录	相关用户
GET	/packages/by-code/{code}	按取件码查询在库包裹	工作人员、管理员
POST	/packages/{id}/shelf	调整货架格口	工作人员、管理员
POST	/packages/{id}/pickup	取件核验并出库	工作人员、管理员
POST	/packages/{id}/return	登记退回	工作人员、管理员
GET	/packages/track/{tracking_no}	按运单号查询本人包裹轨迹	登录用户
GET	/pickup-records	分页查询取件记录	工作人员、管理员
GET	/pickup-records/{id}	取件记录详情	工作人员、管理员

4.2.4 寄件管理（M5）

表 4-5 寄件管理（M5）

方法	路径	说明	权限
GET	/shipments	查询寄件订单，学生仅见本人	登录用户
POST	/shipments	创建寄件订单并估算运费	学生
GET	/shipments/{id}	订单详情	本人、工作人员、管理员
POST	/shipments/{id}/accept	驿站揽收称重，确认运费	工作人员、管理员
POST	/shipments/{id}/ship	登记发出节点	工作人员、管理员
POST	/shipments/{id}/complete	确认完成	工作人员、管理员
POST	/shipments/{id}/cancel	取消订单	本人、管理员
GET	/shipments/{id}/tracks	查询轨迹节点	本人、工作人员、管理员

4.2.5 异常、通知、公告与统计（M6—M8）

表 4-6 异常、通知、公告与统计（M6—M8）

方法	路径	说明	权限
GET	/exceptions	查询异常件记录	工作人员、管理员
POST	/packages/{id}/exception	登记异常件	工作人员、管理员
PATCH	/exceptions/{id}	更新异常处理状态	工作人员、管理员
GET	/notifications	查询本人通知	登录用户
PATCH	/notifications/{id}/read	标记单条已读	登录用户
POST	/notifications/read-all	全部标记已读	登录用户
GET	/notifications/unread-count	未读数量	登录用户
GET	/announcements	公告列表	登录用户
POST	/announcements	发布公告	管理员
PUT	/announcements/{id}	修改公告	管理员
DELETE	/announcements/{id}	下架公告	管理员
GET	/stats/overview	总览指标（在库、今日入库、今日出库、超期）	工作人员、管理员
GET	/stats/packages/trend	收发量趋势，按日或按周	工作人员、管理员
GET	/stats/stations/workload	驿站工作量对比	管理员
GET	/stats/overdue	超期率与超期件分布	工作人员、管理员
GET	/logs	操作日志查询	管理员

4.3 关键接口定义

4.3.1 登录

```
POST /api/v1/auth/login
```

请求体：

```
{
  "account": "2023010101",
  "password": "Student@123",
  "role": "student"
}
```

响应示例：

```
{
  "code": 0,
  "message": "success",
  "data": {
    "access_token": "eyJhbGciOiJIUzI1NiIs...",
    "refresh_token": "eyJhbGciOiJIUzI1NiIs...",
    "token_type": "Bearer",
    "expires_in": 7200,
    "user": {
      "id": 1024,
      "name": "张同学",
      "role": "student",
      "phone": "138****6621"
    }
  },
  "request_id": "c1f7a2e4-9b3d-4a6e-8f21-5d0c7b9e4a10"
}
```

说明：`account` 支持学号、工号或手机号；登录成功后 `last_login_at` 更新；连续 5 次密码错误锁定账号 15 分钟。

4.3.2 快递入库

`POST /api/v1/packages`

请求体：

```
{
  "tracking_no": "SF1234567890123",
  "station_id": 1,
  "recipient_phone": "13800006621",
  "courier_company": "顺丰速运",
  "package_type": "normal",
  "weight": 1.2,
  "shelf_id": null
}
```

说明：`shelf_id` 为空时由系统推荐当前占用率最低的可用货架；`recipient_phone` 用于匹配用户，未匹配时按临时收件人处理。

响应示例：

```
{
  "code": 0,
  "message": "success",
  "data": {
    "id": 5001,
    "tracking_no": "SF1234567890123",
    "pickup_code": "384726",
    "code_expire_at": "2026-11-02T18:30:00+08:00",
    "shelf_id": 12,
    "shelf_code": "A-03",
    "status": "in_stock",
    "stored_at": "2026-10-26T18:30:00+08:00"
  },
  "request_id": "8d2f0c6b-3a41-4d7e-9f55-2b8c1e6a7d33"
}
```

主要错误：`20001` 运单号已存在；`20005` 驿站格口已满；`10001` 手机号格式错误。

4.3.3 取件核验

`POST /api/v1/packages/{id}/pickup`

请求体：

```
{
  "pickup_code": "384726",
  "phone_suffix": "6621",
  "verify_type": "code_phone"
}
```

响应示例：

```
{
  "code": 0,
  "message": "success",
  "data": {
    "package_id": 5001,
    "pickup_record_id": 9001,
    "picked_at": "2026-10-26T19:02:11+08:00",
    "overdue_days": 0,
    "status": "picked"
  },
  "request_id": "5a7e9c21-0f6b-4d38-b1c9-7e2a4f8d6b90"
}
```

主要错误：`20002` 取件码无效或已过期；`20003` 手机号后四位不匹配；`20004` 包裹状态不允许出库。

4.3.4 寄件下单

`POST /api/v1/shipments`

请求体：

```
{
  "station_id": 1,
  "receiver_name": "李同学",
  "receiver_phone": "13900001234",
  "receiver_address": "北京市海淀区某大学 3 号宿舍楼",
  "item_name": "书籍",
  "weight": 2.0,
  "remark": "易碎，请轻放"
}
```

响应示例：

```
{
  "code": 0,
  "message": "success",
  "data": {
    "id": 301,
    "order_no": "S20261026000123",
    "freight": 12.00,
    "status": "pending",
    "created_at": "2026-10-26T18:40:00+08:00"
  },
  "request_id": "b9c4e7a1-2d55-4f0a-9c31-6e8b7d2a4c55"
}
```

运费规则：首重 1 kg 收费 8 元，续重每 1 kg 加收 2 元，不足 1 kg 按 1 kg 计；实际运费以驿站称重为准。

4.4 错误码

表 4-7 错误码

业务码	HTTP 状态	含义	处理建议
0	200 / 201	成功	—
10001	422	参数校验失败	按返回字段提示用户修正
10002	401	未登录或令牌失效	跳转登录或使用刷新令牌
10003	403	无权限访问	隐藏入口，提示联系管理员
10004	404	资源不存在	返回列表页
10005	409	数据状态冲突	刷新后重试
20001	409	运单号已存在	展示原入库记录
20002	400	取件码无效或已过期	引导人工查询
20003	400	手机号后四位不匹配	记录失败次数，提示核对
20004	409	包裹状态不允许当前操作	展示流转记录
20005	409	驿站格口已满	提示扩容或转其他驿站
30001	409	寄件订单状态不允许当前操作	刷新订单状态
50000	500	服务器内部错误	记录 request_id，联系管理员

4.5 接口安全与幂等

1. **认证**：访问令牌有效期 2 小时，刷新令牌有效期 7 天；注销后的令牌写入 Redis 黑名单，剩余有效期过后自动清理。
2. **越权防护**：学生查询包裹时强制追加 `recipient_id = 当前用户`；工作人员只能操作所属驿站的数据；管理员不受驿站限制。
3. **取件码防爆破**：同一驿站 10 分钟内连续 5 次核验失败，暂停该手机号的核验请求 10 分钟，并写入操作日志。
4. **入库幂等**：以 `packages.tracking_no` 唯一约束兜底，重复提交返回 `20001` 与原记录，不产生新数据。
5. **出库幂等**：更新包裹状态时附加 `status IN ('in_stock','overdue')` 条件，并使用行锁；`pickup_records.package_id` 唯一约束防止重复写入取件记录。
6. **传输安全**：生产环境由 Nginx 配置 HTTPS，后端只监听内网端口；跨域仅允许前端域名。

5 数据库设计

5.1 设计策略

1. 数据库使用 MySQL 8.0，存储引擎 InnoDB，字符集 `utf8mb4`，排序规则 `utf8mb4_0900_ai_ci`。
2. 表名与字段名统一使用小写下划线命名，表名用复数形式，如 `packages`。
3. 主键统一为 `BIGINT UNSIGNED` 自增，避免业务主键变更带来的级联修改。
4. 外键字段命名为 `<表名单数>_id`，物理外键仅保留核心业务关系，日志类表不建外键以提升写入性能。
5. 所有业务表包含 `created_at`、`updated_at`，由数据库默认值维护，接口层不手工赋值。
6. 金额使用 `DECIMAL(8,2)`，重量使用 `DECIMAL(6,2)`，禁止使用浮点数存储金额。
7. 状态字段使用 `VARCHAR(16)` 存储枚举值，由后端常量约束，避免 MySQL ENUM 修改成本高。
8. 用户表采用 `status` 字段做逻辑禁用，其余业务表采用物理删除或状态流转，不做逻辑删除。

5.2 E-R 图

系统核心实体的 E-R 图如图 5-1 所示。

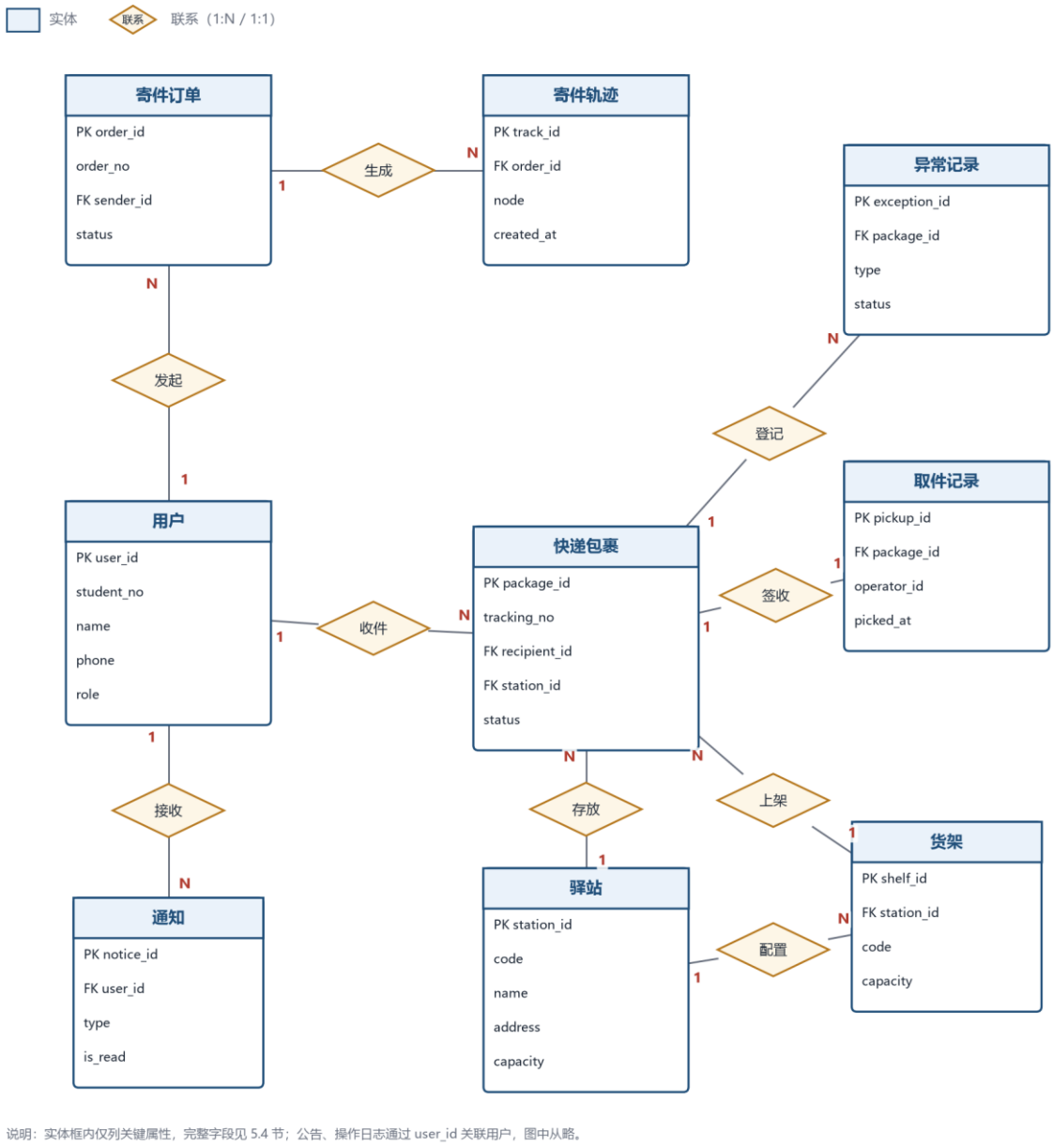


图 5-1 系统 E-R 图

图中实体的关键属性以精简形式列在实体框内，完整字段见 5.4 节。公告与操作日志通过 `user_id` 关联用户实体，为避免连线过多，图中从略。

5.3 实体关系说明

表 5-1 实体关系说明

关系	基数	说明
用户—快递包裹（收件）	1:N	一个用户可以接收多件快递；临时收件人允许 `recipient_id` 为空
驿站—货架（配置）	1:N	一个驿站配置多个货架，货架编号在驿站内唯一
驿站—快递包裹（存放）	1:N	包裹入库时必须归属一个驿站
货架—快递包裹（上架）	1:N	一个格口同一时间只放一件包裹，包裹出库后释放
快递包裹—取件记录（签收）	1:1	一件包裹最多一条成功取件记录，由唯一约束保证
快递包裹—异常记录（登记）	1:N	一件包裹可登记多次异常，如破损后转退回
用户—通知（接收）	1:N	通知按用户维度写入，支持站内信与短信渠道
用户—寄件订单（发起）	1:N	学生可创建多个寄件订单
寄件订单—寄件轨迹（生成）	1:N	订单每流转一个节点写入一条轨迹
用户—公告（发布）	1:N	仅管理员可发布公告
用户—操作日志（产生）	1:N	关键写操作均记录操作人

说明：寄件订单通过 `station_id` 关联驿站，属于“驿站受理寄件”的关系；为使图形保持可读，该连线未在 E-R 图中绘制，关系在表结构中体现。

5.4 数据库表结构

5.4.1 users（用户表）

表 5-2 users（用户表）

字段名	类型	允许空	键与约束	说明
id	BIGINT UNSIGNED	否	PK，自增	用户主键
student_no	VARCHAR(20)	是	UK	学号或工号，管理员可为空
name	VARCHAR(50)	否	—	真实姓名
phone	VARCHAR(11)	否	UK	手机号，登录与取件核验使用
password_hash	VARCHAR(100)	否	—	bcrypt 哈希，禁止存明文
role	VARCHAR(16)	否	IDX	student / staff / admin
college	VARCHAR(100)	是	—	学院或部门
avatar_url	VARCHAR(255)	是	—	头像地址
status	TINYINT	否	IDX	1 正常，0 禁用，默认 1
last_login_at	DATETIME	是	—	最近登录时间
created_at	DATETIME	否	—	创建时间
updated_at	DATETIME	否	—	更新时间

索引：`uk_users_phone(phone)`、`uk_users_student_no(student_no)`、`idx_users_role_status(role, status)`。

5.4.2 stations（驿站表）

表 5-3 stations（驿站表）

字段名	类型	允许空	键与约束	说明
id	BIGINT UNSIGNED	否	PK，自增	驿站主键
code	VARCHAR(20)	否	UK	驿站编码，如 ST-A
name	VARCHAR(50)	否	—	驿站名称
address	VARCHAR(200)	否	—	详细地址
phone	VARCHAR(11)	是	—	联系电话
open_time	TIME	是	—	营业开始时间
close_time	TIME	是	—	营业结束时间
capacity	INT	否	—	总格口数，默认 0
manager_id	BIGINT UNSIGNED	是	FK→users.id	驿站负责人
status	TINYINT	否	IDX	1 营业，0 停用
created_at	DATETIME	否	—	创建时间
updated_at	DATETIME	否	—	更新时间

索引：`uk_stations_code(code)`、`idx_stations_status(status)`。

5.4.3 shelves（货架表）

表 5-4 shelves（货架表）

字段名	类型	允许空	键与约束	说明
id	BIGINT UNSIGNED	否	PK，自增	货架主键
station_id	BIGINT UNSIGNED	否	FK→stations.id	所属驿站
code	VARCHAR(20)	否	UK（联合）	货架编号，如 A-03
zone	VARCHAR(20)	是	—	分区，如 A 区
capacity	INT	否	—	格口数量
used_count	INT	否	—	已用格口数，出库时递减
status	TINYINT	否	—	1 可用，0 停用
created_at	DATETIME	否	—	创建时间
updated_at	DATETIME	否	—	更新时间

索引：`uk_shelves_station_code(station_id, code)`、`idx_shelves_station(station_id)`。

5.4.4 packages（快递包裹表）

表 5-5 packages（快递包裹表）

字段名	类型	允许空	键与约束	说明
id	BIGINT UNSIGNED	否	PK，自增	包裹主键
tracking_no	VARCHAR(40)	否	UK	快递运单号，全局唯一
station_id	BIGINT UNSIGNED	否	FK→stations.id	所属驿站

字段名	类型	允许空	键与约束	说明
shelf_id	BIGINT UNSIGNED	是	FK→shelves.id	存放货架，出库后置空
recipient_id	BIGINT UNSIGNED	是	FK→users.id	收件人，未匹配时为空
recipient_name	VARCHAR(50)	是	—	收件人姓名，来自运单
recipient_phone	VARCHAR(11)	否	IDX	收件人手机号
courier_company	VARCHAR(30)	否	—	快递公司
package_type	VARCHAR(16)	否	—	normal / fragile / large / doc
weight	DECIMAL(6,2)	是	—	重量（kg）
status	VARCHAR(16)	否	IDX	in_stock / picked / overdue / returned / exception
pickup_code	VARCHAR(6)	是	UK（联合）	6 位取件码，站内唯一
code_expire_at	DATETIME	是	IDX	取件码失效时间
stored_at	DATETIME	否	—	入库时间，超期判定基准
picked_at	DATETIME	是	—	取件时间
overdue_days	INT	否	—	超期天数，默认 0
remark	VARCHAR(255)	是	—	备注
created_by	BIGINT UNSIGNED	否	FK→users.id	入库操作员
created_at	DATETIME	否	—	创建时间
updated_at	DATETIME	否	—	更新时间

索引：uk_packages_tracking_no(tracking_no)、uk_packages_station_code(station_id, pickup_code)、idx_packages_recipient_status(recipient_id, status)、idx_packages_status_expire(status, code_expire_at)、idx_packages_shelf(shelf_id)。

5.4.5 pickup_records（取件记录表）

表 5-6 pickup_records（取件记录表）

字段名	类型	允许空	键与约束	说明
id	BIGINT UNSIGNED	否	PK，自增	记录主键
package_id	BIGINT UNSIGNED	否	FK→packages.id，UK	一件包裹仅一条成功记录
operator_id	BIGINT UNSIGNED	否	FK→users.id	出库操作员
pickup_code	VARCHAR(6)	否	—	核验时的取件码快照
phone_suffix	CHAR(4)	否	—	手机号后四位快照
verify_type	VARCHAR(16)	否	—	code_phone / manual
picked_at	DATETIME	否	IDX	取件时间
remark	VARCHAR(200)	是	—	备注
created_at	DATETIME	否	—	创建时间

索引：uk_pickup_records_package(package_id)、idx_pickup_records_time(picked_at)。

5.4.6 shipment_orders（寄件订单表）

表 5-7 shipment_orders（寄件订单表）

字段名	类型	允许空	键与约束	说明
id	BIGINT UNSIGNED	否	PK，自增	订单主键
order_no	VARCHAR(32)	否	UK	订单号，S + 日期 + 序号
sender_id	BIGINT UNSIGNED	否	FK→users.id	寄件人
station_id	BIGINT UNSIGNED	否	FK→stations.id	受理驿站
receiver_name	VARCHAR(50)	否	—	收件人姓名
receiver_phone	VARCHAR(11)	否	—	收件人电话
receiver_address	VARCHAR(200)	否	—	收件地址
courier_company	VARCHAR(30)	是	—	快递公司，揽收时指定
item_name	VARCHAR(100)	否	—	物品名称
weight	DECIMAL(6,2)	是	—	实际重量，揽收时确认
freight	DECIMAL(8,2)	是	—	运费
status	VARCHAR(16)	否	IDX	pending / accepted / shipped / completed / cancelled
remark	VARCHAR(255)	是	—	备注
created_at	DATETIME	否	—	下单时间
updated_at	DATETIME	否	—	更新时间

索引：`uk_shipment_orders_no(order_no)`、`idx_shipment_sender(sender_id)`、`idx_shipment_station_status(station_id, status)`。

5.4.7 shipment_tracks（寄件轨迹表）

表 5-8 shipment_tracks（寄件轨迹表）

字段名	类型	允许空	键与约束	说明
id	BIGINT UNSIGNED	否	PK，自增	轨迹主键
order_id	BIGINT UNSIGNED	否	FK→shipment_orders.id	所属订单
node	VARCHAR(50)	否	—	节点名称，如“驿站揽收”
status	VARCHAR(16)	否	—	节点对应的订单状态
description	VARCHAR(200)	是	—	节点描述
operator_id	BIGINT UNSIGNED	是	FK→users.id	操作人
occurred_at	DATETIME	否	IDX	发生时间
created_at	DATETIME	否	—	创建时间

索引：`idx_shipment_tracks_order_time(order_id, occurred_at)`。

5.4.8 notifications（通知表）

表 5-9 notifications（通知表）

字段名	类型	允许空	键与约束	说明
id	BIGINT UNSIGNED	否	PK，自增	通知主键
user_id	BIGINT UNSIGNED	否	FK→users.id	接收人
type	VARCHAR(16)	否	IDX	pickup / overdue / shipment / system
title	VARCHAR(100)	否	—	通知标题
content	VARCHAR(500)	否	—	通知内容
package_id	BIGINT UNSIGNED	是	—	关联包裹
order_id	BIGINT UNSIGNED	是	—	关联寄件订单
channel	VARCHAR(16)	否	—	inapp / sms
is_read	TINYINT	否	IDX	0 未读，1 已读
sent_at	DATETIME	是	—	发送时间
created_at	DATETIME	否	—	创建时间

索引：`idx_notifications_user_read(user_id, is_read)`、`idx_notifications_type(type)`。

5.4.9 announcements（公告表）

表 5-10 announcements（公告表）

字段名	类型	允许空	键与约束	说明
id	BIGINT UNSIGNED	否	PK，自增	公告主键
title	VARCHAR(100)	否	—	公告标题
content	TEXT	否	—	公告正文
publisher_id	BIGINT UNSIGNED	否	FK→users.id	发布人
status	TINYINT	否	IDX	1 已发布，0 草稿或下架
publish_at	DATETIME	是	IDX	发布时间
created_at	DATETIME	否	—	创建时间
updated_at	DATETIME	否	—	更新时间

索引：`idx_announcements_status_time(status, publish_at)`。

5.4.10 exception_records（异常记录表）

表 5-11 exception_records（异常记录表）

字段名	类型	允许空	键与约束	说明
id	BIGINT UNSIGNED	否	PK，自增	异常主键
package_id	BIGINT UNSIGNED	否	FK→packages.id	关联包裹
type	VARCHAR(16)	否	IDX	damaged / lost / wrong_station / returned / other
description	VARCHAR(300)	否	—	异常描述
handler_id	BIGINT UNSIGNED	是	FK→users.id	处理人
status	VARCHAR(16)	否	IDX	open / processing / resolved
resolved_at	DATETIME	是	—	处理完成时间
created_at	DATETIME	否	—	创建时间
updated_at	DATETIME	否	—	更新时间

索引：`idx_exceptions_package(package_id)`、`idx_exceptions_status(status)`。

5.4.11 operation_logs（操作日志表）

表 5-12 operation_logs（操作日志表）

字段名	类型	允许空	键与约束	说明
id	BIGINT UNSIGNED	否	PK，自增	日志主键
user_id	BIGINT UNSIGNED	是	IDX	操作人，系统任务为空
module	VARCHAR(30)	否	IDX	模块标识，如 package
action	VARCHAR(50)	否	—	动作，如 inbound、pickup
target_type	VARCHAR(30)	是	—	目标类型
target_id	BIGINT UNSIGNED	是	—	目标主键
detail	JSON	是	—	关键参数快照
ip	VARCHAR(45)	是	—	客户端 IP，兼容 IPv6
created_at	DATETIME	否	IDX	操作时间

索引：`idx_logs_user_time(user_id, created_at)`、`idx_logs_module_time(module, created_at)`。

5.5 数据字典

表 5-13 数据字典

字段	取值	含义
users.role	student / staff / admin	学生、驿站工作人员、管理员
users.status	1 / 0	正常 / 禁用
packages.status	in_stock / picked / overdue / returned / exception	在库、已取件、超期、已退回、异常
packages.package_type	normal / fragile / large / doc	普通、易碎、大件、文件

字段	取值	含义
pickup_records.verify_type	code_phone / manual	取件码+手机号核验 / 人工核验
shipment_orders.status	pending / accepted / shipped / completed / cancelled	待揽收、已揽收、已发出、已完成、已取消
exception_records.type	damaged / lost / wrong_station / returned / other	破损、丢失、错分、退回、其他
exception_records.status	open / processing / resolved	待处理、处理中、已解决
notifications.type	pickup / overdue / shipment / system	取件、超期、寄件、系统
notifications.channel	inapp / sms	站内信、短信（模拟）

5.6 索引与约束说明

1. **唯一约束**：`packages.tracking_no` 保证运单号不重复；`(station_id, pickup_code)` 保证取件码站内唯一；`pickup_records.package_id` 保证一件包裹只有一条成功取件记录，是出库幂等的最后一道防线。
2. **组合索引**：`(recipient_id, status)` 支撑学生端“我的包裹”按状态筛选；`(status, code_expire_at)` 支撑超期扫描任务；`(station_id, status)` 支撑驿站工作台列表。
3. **外键策略**：业务表外键默认 `ON DELETE RESTRICT`，防止误删驿站或用户导致包裹数据悬空；`shipment_tracks`、`notifications` 等流水表仅在应用层校验，减少写放大。
4. **计数冗余**：`shelves.used_count` 为冗余字段，入库加一、出库减一，均在同一事务内完成；每日凌晨提供校准任务，按实际在库包裹数修正。
5. **时间字段**：`stored_at` 是超期判定的唯一基准，`picked_at` 仅在出库时写入，二者均可为空的情况在接口层校验。

6 详细设计

本章选取“入库登记”与“取件核验”两个核心模块进行详细设计，每个模块给出功能说明、程序流程图、N-S 图、关键伪代码与异常处理。其余模块的处理要点在 6.3 节汇总。

6.1 入库登记模块（M3）

6.1.1 功能说明

入库登记负责将到达驿站的快递写入系统，是收件业务的起点。模块输入为运单号、驿站、收件人手机号、快递公司与包裹属性；输出为包裹记录、取件码、上架格口与取件通知。

关键规则：

1. 运单号全局唯一，重复入库直接拒绝并返回原记录。
2. 收件人手机号用于匹配系统用户，未匹配时登记为临时收件人，取件时仅核对手机号后四位。
3. 货架由工作人员指定或系统推荐，推荐策略为“可用格口数最多且状态正常”的货架。
4. 取件码为 6 位随机数字，在同一驿站内唯一，有效期默认 7 天，可在配置文件中调整。
5. 入库成功后包裹状态为 `in_stock`，同时写入操作日志并生成通知。

6.1.2 程序流程图

入库登记的程序流程如图 6-1 所示。

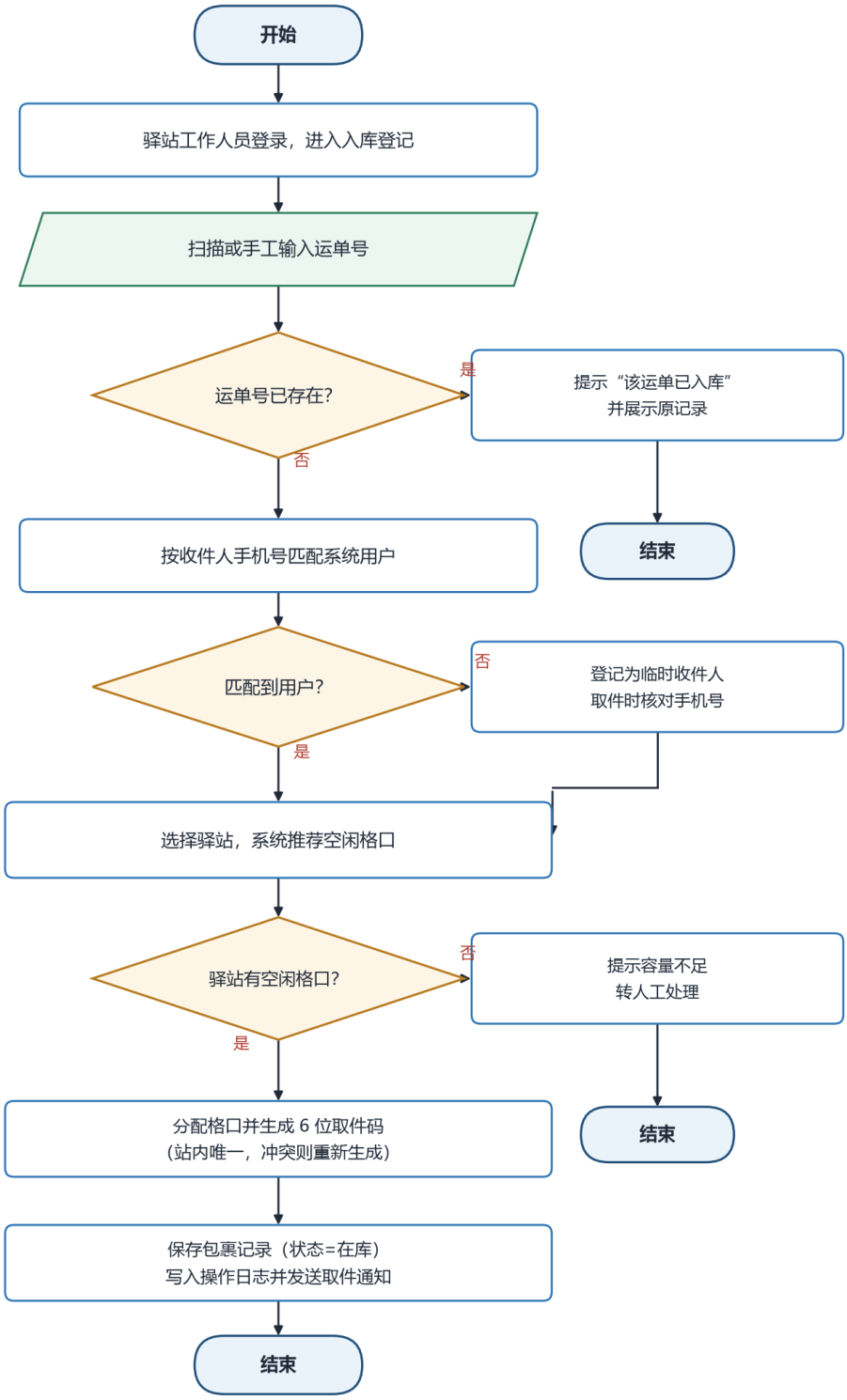


图 6-1 入库登记程序流程图

6.1.3 N-S 图

入库登记的 N-S 图如图 6-2 所示。



图 6-2 入库登记模块 N-S 图

6.1.4 关键处理逻辑

```
输入：tracking_no, station_id, recipient_phone, courier_company,
      package_type, weight, shelf_id(可选)

1. 校验参数格式（运单号长度、手机号 11 位、驿站存在且营业）
2. 查询 packages WHERE tracking_no = :tracking_no
   若存在 → 返回 20001，附带原包裹 id 与入库时间
3. 按 recipient_phone 查询 users
   若唯一匹配 → recipient_id = user.id
   若未匹配   → recipient_id = NULL，recipient_name 取运单信息
4. 开启事务
5. 若 shelf_id 为空：
       SELECT * FROM shelves
       WHERE station_id = :station_id AND status = 1
         AND used_count < capacity
       ORDER BY (capacity - used_count) DESC LIMIT 1
   若仍为空 → 返回 20005，事务回滚
6. 循环生成取件码（最多 5 次）：
       code = 6 位随机数字
       若 (station_id, code) 未被有效包裹占用 → 跳出循环
   若 5 次均冲突 → 返回 50000 并记录告警
7. INSERT packages(status='in_stock', stored_at=NOW(),
                   code_expire_at=NOW()+7d, ...)
8. UPDATE shelves SET used_count = used_count + 1 WHERE id = :shelf_id
9. INSERT notifications(type='pickup', channel='inapp')
10. INSERT operation_logs(module='package', action='inbound')
11. 提交事务，返回包裹信息与取件码
```

事务边界为第 5 步至第 10 步，任一步失败整体回滚，避免出现“包裹已写入但格口未占用”的脏数据。

6.1.5 异常处理

表 6-1 异常处理

异常场景	检测方式	处理策略	返回码
运单号重复	唯一索引 + 前置查询	展示原入库记录，不新建	20001
手机号格式错误	Pydantic 正则校验	前端标红提示	10001
驿站停用	查询驿站 status	禁止入库，提示联系管理员	10005
格口已满	推荐查询无结果	提示扩容或转其他驿站	20005
取件码连续冲突	循环 5 次仍冲突	回滚并记录告警日志	50000
并发重复入库	唯一索引冲突	捕获 IntegrityError，转为 20001	20001

6.2 取件核验模块（M4）

6.2.1 功能说明

取件核验负责验证取件人身份并完成出库，是收件业务的终点。模块输入为取件码、手机号后四位与核验方式；输出为取件记录、包裹状态变更、格口释放与通知。

关键规则：

1. 取件码必须在所属驿站内存在、未失效，且包裹状态为 `in_stock` 或 `overdue`。
2. 手机号后四位必须与 `packages.recipient_phone` 的后四位一致。
3. 同一件包裹只能成功出库一次，重复提交返回 `20004`。
4. 出库时释放格口，`shelves.used_count` 减一，不允许出现负数。
5. 超期件出库时记录超期天数，供统计模块使用。

6.2.2 程序流程图

取件核验的程序流程如图 6-3 所示。

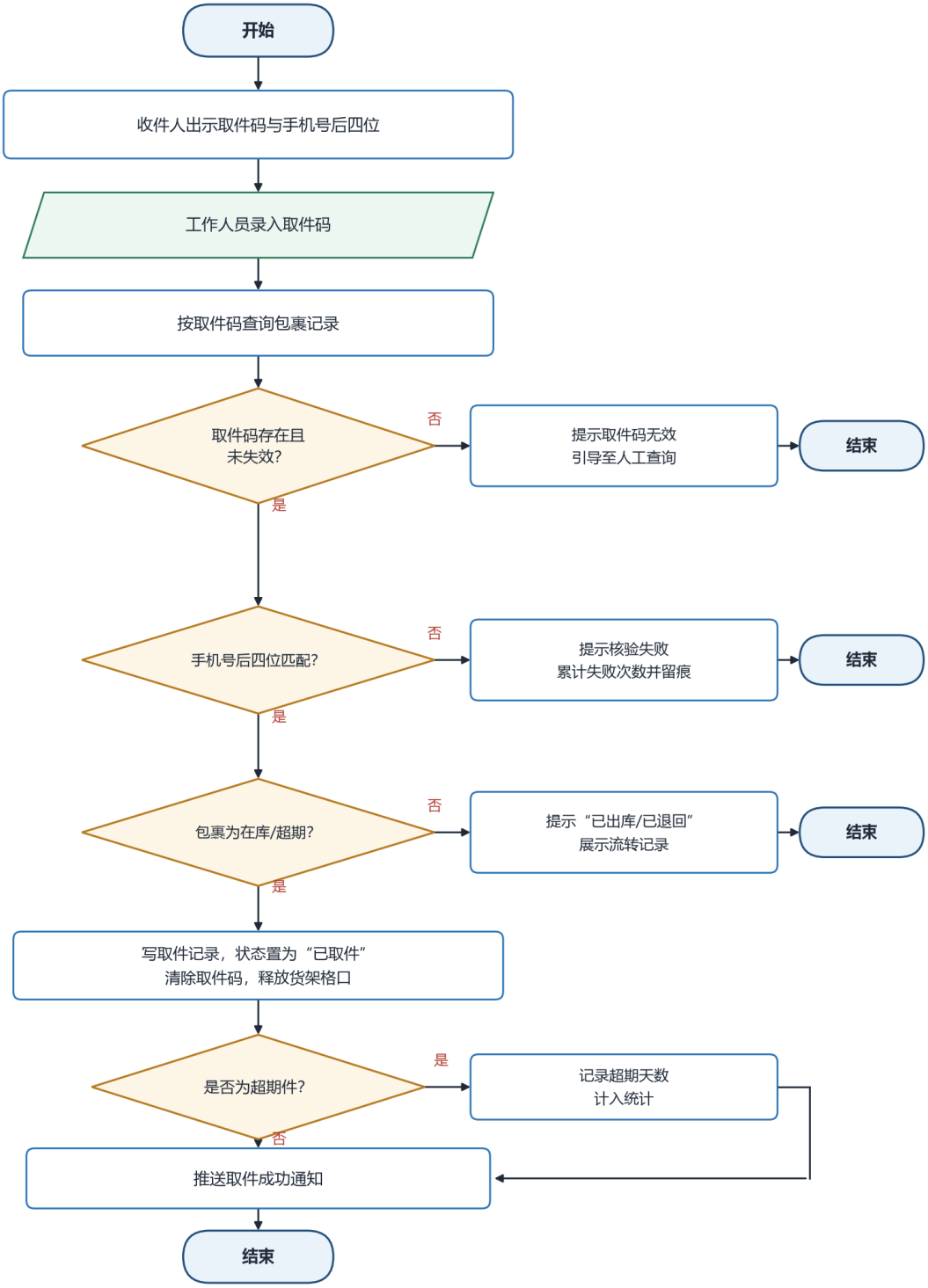


图 6-3 取件核验程序流程图

6.2.3 N-S 图

取件核验的 N-S 图如图 6-4 所示。



图 6-4 取件核验模块 N-S 图

6.2.4 关键处理逻辑

```
输入：package_id, pickup_code, phone_suffix, operator_id

1. 开启事务
2. SELECT * FROM packages WHERE id = :package_id FOR UPDATE
   若不存在 → 返回 10004
3. 校验 status ∈ {in_stock, overdue}
   否则 → 返回 20004，附带当前状态
4. 校验 pickup_code 一致且 code_expire_at > NOW()
   否则 → 失败次数 +1，返回 20002
5. 校验 RIGHT(recipient_phone, 4) = :phone_suffix
   否则 → 失败次数 +1，返回 20003
6. INSERT pickup_records(package_id, operator_id, pickup_code,
                         phone_suffix, verify_type, picked_at=NOW())
7. UPDATE packages
      SET status='picked', picked_at=NOW(), pickup_code=NULL,
          shelf_id=NULL, overdue_days=:computed
      WHERE id=:package_id AND status IN ('in_stock','overdue')
   若影响行数 = 0 → 回滚，返回 20004
8. UPDATE shelves SET used_count = GREATEST(used_count-1, 0)
      WHERE id = :old_shelf_id
9. INSERT notifications(type='pickup', title='取件成功')
10. INSERT operation_logs(module='package', action='pickup')
11. 提交事务
```

其中 overdue_days 按 DATEDIFF(NOW(), stored_at) - overdue_threshold 计算，结果小于 0 时取 0。

6.2.5 异常处理

表 6-2 异常处理

异常场景	检测方式	处理策略	返回码
取件码不存在	按 id 查询后比对	提示核对，记录失败次数	20002
取件码过期	code_expire_at 比较	提示已过期，引导人工查询	20002
手机号不匹配	后四位比较	提示核对，累计失败次数	20003
包裹已出库	状态判断	展示取件时间与操作员	20004
重复提交	状态条件更新 + 唯一约束	返回已出库结果，不重复写记录	20004
格口计数异常	更新时 GREATEST 保护	记录告警，由校准任务修正	—

6.3 其他模块关键逻辑

表 6-3 其他模块关键逻辑

模块	关键处理逻辑	事务边界
M1 登录	按账号查用户→bcrypt 校验→签发双令牌→更新登录时间；连续失败 5 次锁定 15 分钟	更新登录时间与失败计数
M2 货架维护	新增货架时校验 `(station_id, code)` 唯一；删除前检查 `used_count = 0`	单表单条
M5 寄件下单	按重量计算运费→生成订单号→写订单→写“待揽收”轨迹	订单与首条轨迹
M5 揽收	校验状态为 pending→更新重量与运费→状态置 accepted→写轨迹	订单与轨迹
M6 超期判定	每日 02:00 扫描 `status='in_stock' AND stored_at < NOW()-3d`，批量置为 overdue 并写提醒	批量更新 + 通知
M6 退回	校验状态→置 returned→释放格口→写异常记录→通知	包裹、货架、异常、通知
M7 通知	入库、出库、超期事件写入通知表；未读数量走 Redis 计数	与业务同事务
M8 统计	按日聚合入库、出库、超期数量，结果缓存 5 分钟	只读

7 界面设计

7.1 页面结构

系统按角色划分为三个工作台，页面清单如下。

表 7-1 页面结构

端	页面	路由	说明
公共	登录	/login	账号密码登录，按角色跳转
公共	注册	/register	学生自助注册
学生端	我的包裹	/student/packages	列表、状态筛选、取件码展示
学生端	包裹详情	/student/packages/:id	运单信息、取件码、驿站地址
学生端	寄件下单	/student/shipments/new	填写寄件信息、运费估算
学生端	我的寄件	/student/shipments	订单列表与轨迹
学生端	消息中心	/student/notifications	通知列表、已读标记
驿站端	入库登记	/staff/inbound	扫码输入、格口分配、取件码展示
驿站端	取件核验	/staff/pickup	取件码输入、核验结果、出库确认
驿站端	在库包裹	/staff/packages	列表、筛选、退回与异常登记
驿站端	寄件揽收	/staff/shipments	待揽收订单、称重计费、轨迹登记
驿站端	货架管理	/staff/shelves	货架占用率、格口调整
管理端	用户管理	/admin/users	用户列表、新增、禁用
管理端	驿站管理	/admin/stations	驿站与货架维护
管理端	统计报表	/admin/stats	收发量趋势、驿站工作量、超期率
管理端	公告管理	/admin/announcements	发布、编辑、下架
管理端	操作日志	/admin/logs	按模块、操作人、时间筛选

7.2 路由与权限

前端使用 Vue Router 的全局前置守卫实现权限控制：

1. 路由 `meta.roles` 声明允许访问的角色，`meta.requiresAuth` 声明是否需要登录。
2. 守卫读取 Pinia 中的用户信息与访问令牌；未登录跳转 `/login`，角色不匹配跳转 403 页面。
3. 访问令牌过期时先调用 `/auth/refresh` 静默续期，续期失败再跳转登录。
4. 菜单根据角色动态渲染，学生端不显示入库、核验入口。

7.3 关键交互

1. **入库页**：运单号输入框自动聚焦，扫码枪回车即提交；提交成功后取件码以大字号展示，并提供“复制并发送”按钮。

2. **取件页**：取件码输入框支持数字键盘；核验成功后整屏显示绿色成功提示与包裹信息，3 秒后自动返回待核验状态。
3. **超期标识**：列表中超期件整行使用浅红底色，状态标签显示“超期 N 天”。
4. **空状态**：所有列表提供空状态插画与操作引导，避免出现空白页面。
5. **响应式**：驿站工作台按 1366×768 设计，表格支持横向滚动；学生端适配手机浏览器。

8 部署与运行设计

8.1 部署拓扑

系统采用 Docker Compose 编排，包含前端、后端、数据库与缓存四个服务，部署拓扑如图 8-1 所示。

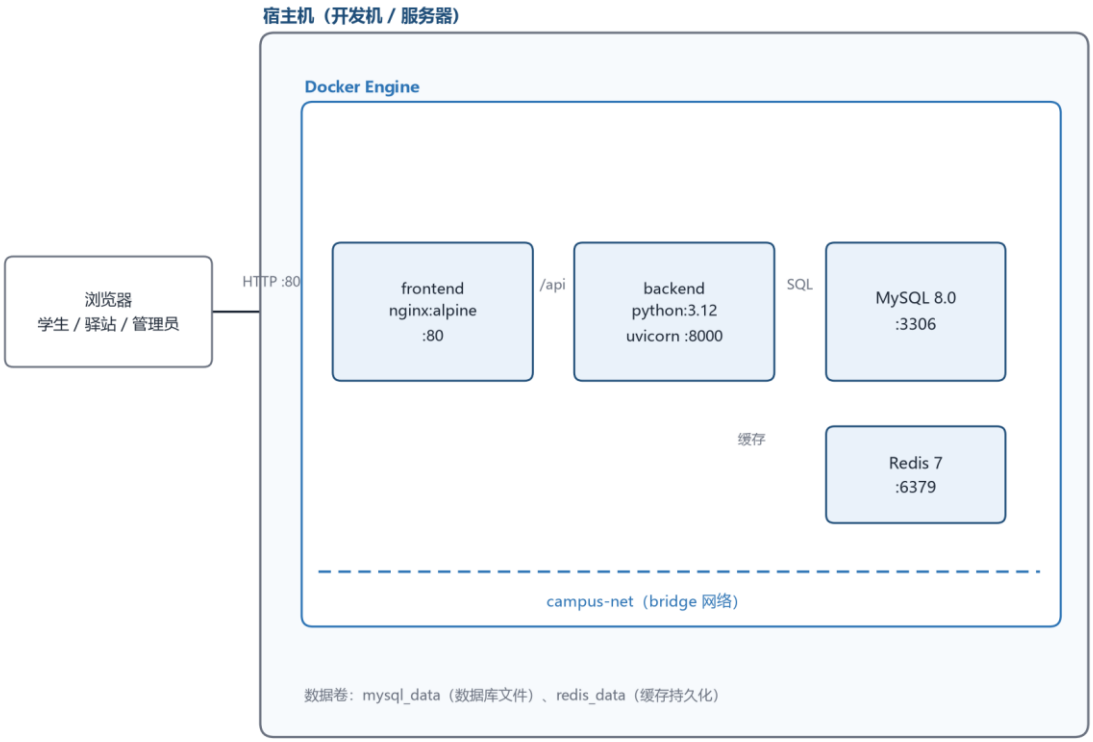


图 8-1 Docker 部署架构图

8.2 Docker 编排设计

`deploy/docker-compose.yml` 定义四个服务，统一加入 `campus-net` 网络：

表 8-1 Docker 编排设计

服务	镜像 / 构建	端口映射	数据卷	依赖
mysql	mysql:8.0	3306:3306	mysql_data:/var/lib/mysql	—
redis	redis:7-alpine	6379:6379	redis_data:/data	—
backend	由 backend/Dockerfile 构建	8000:8000	—	mysql healthy、redis healthy
frontend	由 frontend/Dockerfile 构建（Nginx）	80:80	—	backend

关键配置：

```
services:
  mysql:
    image: mysql:8.0
    environment:
      MYSQL_ROOT_PASSWORD: ${MYSQL_ROOT_PASSWORD}
      MYSQL_DATABASE: ${MYSQL_DATABASE}
      MYSQL_USER: ${MYSQL_USER}
      MYSQL_PASSWORD: ${MYSQL_PASSWORD}
      TZ: Asia/Shanghai
    command: --character-set-server=utf8mb4 --collation-server=utf8mb4_0900_ai_ci
    volumes:
      - mysql_data:/var/lib/mysql
      - ./mysql/init:/docker-entrypoint-initdb.d:ro
    healthcheck:
      test: ["CMD", "mysqladmin", "ping", "-h", "localhost"]
      interval: 10s
      timeout: 5s
      retries: 10

  backend:
    build: ../backend
    environment:
      DATABASE_URL:
mysql+pymysql://campus:${MYSQL_PASSWORD}@mysql:3306/campus_express
      REDIS_URL: redis://redis:6379/0
      JWT_SECRET: ${JWT_SECRET}
    depends_on:
      mysql:
        condition: service_healthy
      redis:
        condition: service_healthy
```

设计要点：

1. 数据库使用官方 `mysql:8.0` 镜像，不在本地安装 MySQL，保证组内数据目录与版本一致。
2. 后端容器启动时先执行 `alembic upgrade head`，再启动 `uvicorn`，避免表结构缺失。
3. 前端构建产物由 Nginx 托管，`/api` 反向代理到 `backend:8000`，避免跨域。
4. 数据卷 `mysql_data`、`redis_data` 持久化，执行 `docker compose down` 不丢数据；如需重置，显式删除卷。

8.3 本地开发启动

本地开发仅将数据库与缓存放入 Docker，前后端在宿主机运行，便于热更新与调试。

```
# 1. 启动基础设施
docker compose -f deploy/docker-compose.infra.yml up -d

# 2. 后端
cd backend
python -m venv .venv
source .venv/Scripts/activate        # Windows Git Bash
pip install -r requirements.txt
cp .env.example .env
alembic upgrade head
uvicorn app.main:app --reload --port 8000

# 3. 前端
cd frontend
npm install
npm run dev
```

`deploy/docker-compose.infra.yml` 只包含 mysql 与 redis，端口映射到宿主机，便于使用 Navicat 等工具连接。前端 `vite.config.js` 配置代理：

```
server: {
  port: 5173,
  proxy: {
    '/api': { target: 'http://localhost:8000', changeOrigin: true }
  }
}
```

8.4 配置项

表 8-2 配置项

变量	示例	说明
MYSQL_ROOT_PASSWORD	change_me_root	数据库 root 口令，仅容器初始化使用
MYSQL_DATABASE	campus_express	业务库名
MYSQL_USER	campus	业务账号
MYSQL_PASSWORD	change_me	业务账号口令
DATABASE_URL	mysql+pymysql://campus:change_me@mysql:3306/campus_express	后端连接串
REDIS_URL	redis://redis:6379/0	缓存连接串
JWT_SECRET	随机 32 位字符串	令牌签名密钥，禁止提交到仓库
ACCESS_TOKEN_EXPIRE_MINUTES	120	访问令牌有效期
REFRESH_TOKEN_EXPIRE_DAYS	7	刷新令牌有效期
PICKUP_CODE_EXPIRE_DAYS	7	取件码有效期
OVERDUE_THRESHOLD_DAYS	3	超期判定阈值
TZ	Asia/Shanghai	容器时区

8.5 初始化与迁移

1. 首次启动由 `deploy/mysql/init/01-init.sql` 创建数据库与业务账号，并设置字符集。

2. 表结构由 Alembic 管理：开发阶段每次模型变更执行 `alembic revision --autogenerate -m "描述"`，评审后执行 `alembic upgrade head`。
3. 初始化脚本 `scripts/seed.py` 写入演示数据：1 个管理员、2 个驿站、8 个货架、20 件不同状态的包裹与 5 条寄件订单，便于答辩演示。
4. 生产环境禁止使用 `Base.metadata.create_all()` 建表，统一走迁移脚本。

9 非功能性设计

9.1 安全设计

1. **密码安全**：使用 bcrypt（cost=12）哈希存储，登录失败提示统一为“账号或密码错误”，不泄露账号是否存在。
2. **令牌安全**：JWT 使用 HS256 签名，密钥从环境变量读取；访问令牌短有效期，刷新令牌可撤销；注销写入 Redis 黑名单。
3. **权限控制**：后端每个接口声明所需角色，数据查询强制附加归属条件，前端隐藏无权入口仅作为体验优化。
4. **取件安全**：取件码 + 手机号后四位双重核验；连续失败限流；取件记录保存核验方式与操作员，便于追溯。
5. **输入安全**：所有入参经 Pydantic 校验；ORM 参数化查询防 SQL 注入；公告正文做 XSS 过滤。
6. **敏感信息**：手机号在列表页脱敏展示（`138****6621`），日志中不记录密码与完整令牌。

9.2 性能设计

表 9-1 性能设计

指标	目标值	保障措施
登录响应	< 500 ms	手机号唯一索引，bcrypt 校验在 100 ms 内
包裹列表	< 300 ms（1 万条数据）	组合索引 + 分页，禁止全表扫描
入库提交	< 400 ms	唯一索引判重，货架推荐走覆盖索引
取件核验	< 300 ms	行锁范围小，事务内仅 4 次写操作
统计报表	< 2 s	按日聚合 + Redis 缓存 5 分钟

前端优化：路由懒加载、Element Plus 按需引入、列表虚拟滚动（超过 200 条时启用）、图片懒加载。

9.3 可靠性与容错

1. 所有写操作使用数据库事务，包裹、货架计数、取件记录、通知保持一致。
2. Docker 服务配置 `restart: unless-stopped`，数据库 healthcheck 通过后才启动后端。

3. 后端提供 `/health` 健康检查接口，返回数据库与 Redis 连接状态。
4. 每日凌晨执行格口计数校准任务，修正因异常中断导致的 `used_count` 偏差。
5. 数据库每周全量备份，备份文件保留 4 份，存放在宿主机挂载目录。

9.4 日志与审计

1. 应用日志按级别输出到标准输出，由 Docker 日志驱动收集，格式为 JSON，包含 `request_id`、用户、路径、耗时。
2. 操作日志记录所有写操作：入库、出库、退回、异常登记、用户变更、公告发布。
3. 日志保留 30 天，超期归档；管理员可在系统中按模块、操作人、时间检索。
4. 关键异常（取件码连续冲突、格口计数为负）触发告警日志，便于课程演示时排查。

9.5 兼容性

1. 浏览器：Chrome 100+、Edge 100+、Firefox 100+，不支持 IE。
2. 分辨率：驿站工作台最低 1366×768，学生端支持 375 px 宽度。
3. 时区：统一使用 `Asia/Shanghai`，数据库、容器、前端展示保持一致。

10 交付物与实施计划

10.1 课程交付物对照

表 10-1 课程交付物对照

阶段	课程要求	本文档对应内容	交付文件
需求分析	业务流程图	3.5 核心业务流程	docs/交付物/业务流程图.docx
需求分析	数据分析表	5.4 数据库表结构、5.5 数据字典	docs/交付物/数据分析表.xlsx
需求分析	功能分析表	3.4 模块清单与职责	docs/交付物/功能分析表.xlsx
需求分析	界面原型	7.1 页面结构、7.3 关键交互	界面原型图（docx，后续提交）
系统设计	模块结构图	3.3 模块结构图	本文档图 3-2
系统设计	E-R 图	5.2 E-R 图	本文档图 5-1
系统设计	数据库表结构	5.4 数据库表结构	本文档 5.4 节 + docs/交付物/数据库表结构.xlsx
详细设计	程序流程图/N-S 图（至少两个模块）	6.1.2、6.1.3、6.2.2、6.2.3	本文档图 6-1 至图 6-4
系统实现	源代码	3.2 技术架构、第 8 章部署设计	源代码压缩包（zip）
系统测试	测试用例与 BUG 清单	6.1.5、6.2.5 异常场景	测试用例 xlsx（后续提交）
项目验收	项目分析总结报告	—	docx（后续提交）
项目管理	项目计划书、周报	10.3 实施计划	docx（后续提交）

10.2 仓库目录

```
Campus_Express/
├── backend/                  # FastAPI 服务
│   ├── app/
│   │   ├── api/v1/           # 路由：auth、users、stations、packages...
│   │   ├── core/             # 配置、安全、依赖、异常处理
│   │   ├── models/           # SQLAlchemy 模型
│   │   ├── schemas/          # Pydantic 模型
│   │   ├── services/         # 业务逻辑
│   │   └── main.py
│   ├── alembic/              # 迁移脚本
│   ├── tests/
│   ├── requirements.txt
│   └── Dockerfile
├── frontend/                 # Vue 3 工程
│   ├── src/{api,components,layouts,router,stores,views}
│   ├── vite.config.js
│   └── Dockerfile
├── deploy/                   # 编排与环境配置
│   ├── docker-compose.yml
│   ├── docker-compose.infra.yml
│   ├── .env.example
│   └── mysql/init/
├── docs/                     # 设计文档与图表
│   ├── 系统设计文档.md
│   ├── 系统设计文档.docx
│   └── assets/
├── scripts/                  # 绘图、文档生成、种子数据脚本
└── README.md
```

10.3 后续阶段计划

表 10-2 后续阶段计划

周次	阶段	主要任务	产出
第 8 周	需求分析	业务流程梳理、数据分析、功能分析、原型设计	业务流程图、xlsx 表单、原型图
第 9 周	系统设计	总体结构、数据库、接口设计	本文档评审通过
第 10 周	详细设计	核心模块流程图、N-S 图	详细设计章节定稿
第 11—12 周	编码（一）	后端框架、认证、驿站货架、入库	可登录、可入库
第 13 周	编码（二）	取件出库、寄件、通知、统计	主流程贯通
第 14 周	联调与测试	前后端联调、测试用例、BUG 修复	测试用例与 BUG 清单
第 15 周	验收准备	部署演练、演示脚本、总结报告	答辩材料与总结报告

附录 A 包裹状态机

表 A-1 包裹状态机

当前状态	触发事件	目标状态	说明
—	入库登记	in_stock	写入取件码与入库时间
in_stock	取件核验通过	picked	写取件记录，释放格口
in_stock	超过 3 天未取	overdue	定时任务批量更新并提醒
overdue	取件核验通过	picked	记录超期天数
in_stock / overdue	登记退回	returned	释放格口，写异常记录
in_stock / overdue	登记异常	exception	破损、丢失等，需人工处理
exception	处理完成且退回	returned	异常关闭
picked / returned	—	终态	不允许再次流转

附录 B 设计待确认事项

以下参数已在设计中给出默认值，编码前需与组内成员确认：

表 B-1 设计待确认事项

事项	当前设计	需确认内容
取件码有效期	7 天	是否与驿站实际清理周期一致
超期判定阈值	3 天	是否按快递公司或包裹类型区分
超期退回阈值	7 天	是否需要二次提醒
工作人员归属	一人一驿站	是否支持一人多驿站轮班
短信通道	站内信 + 日志模拟	答辩演示是否需要真实短信
运费规则	首重 8 元，续重 2 元/kg	是否按快递公司差异化定价